Getting Started with Streamlit

Build a Python application, make it responsive, and use it to manage a large collection of JSON records.

Streamlit is one of many ways to build an application interface. Our focus is the data operations underneath that interface: retrieving, filtering, creating, updating, and deleting records. Later, a database can replace our JSON storage without changing the purpose of those operations.

1. Set up your project

You need Python 3.11 or later, a terminal, and a text editor. Create the files in this tutorial yourself by entering the code shown. Run terminal commands from the project directory.

```
mkdir streamlit-lab
cd streamlit-lab
python -m venv .venv
```

Activate the environment on macOS or Linux:

```
source .venv/bin/activate
```

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Install the packages:

```
python -m pip install streamlit pandas
```

If your system uses `python3`, substitute it for `python` when creating the environment. Once the environment is active, `python` should refer to that environment.

2. Your first application

Create `app.py`:

```
import streamlit as st

st.title("My first Streamlit application")
st.write("A Python script can produce a web interface.")
st.markdown("**Change the script, save it, and watch the page update.**")
```

Start the server:

```
python -m streamlit run app.py
```

Open the local URL printed in the terminal. Leave the terminal running while you work. Stop the server with Ctrl+C.

Add input

Replace `app.py` with:

```
import streamlit as st

st.title("Hello, Streamlit")
name = st.text_input("Your name")
count = st.slider("How many greetings?", 1, 5, 1)

if name:
    for _ in range(count):
        st.write(f"Hello, {name}!")
else:
    st.info("Enter your name to begin.")
```

Widget calls both display controls and return their current values. We can use those values in ordinary Python expressions and loops.

Understand reruns

Think of a change to a widget as a request from the browser to redraw the page using the new value. When you move a slider and release it, the browser sends the new slider value to the Streamlit server. The server records that value for your session and reruns the Python script from the top. The widget call returns the new value, so the Python statements that follow can use it to produce the updated page.

Try adding this immediately after the import:

```
print("The script is running")
```

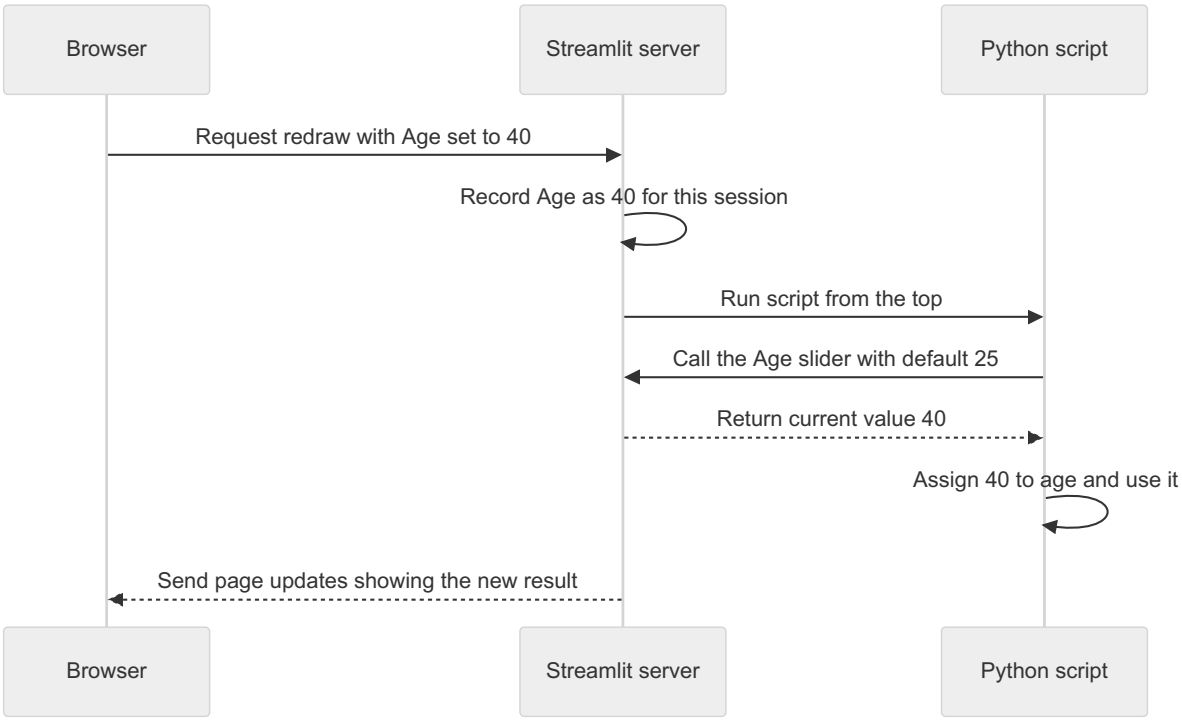
Watch the terminal as you change the slider. `print()` writes to the server terminal; `st.write()` writes to the browser.

A widget call displays a control and returns its value

Consider this call:

```
age = st.slider("Age", 0, 100, 25)
```

On the first run, the slider displays 25 and the call assigns 25 to `age`. Now move the slider to 40. In effect, the browser asks, “Redraw this page with Age set to 40.” On the new run, Python reaches exactly the same line. This time the slider displays 40 and the call assigns 40 to `age`. The default of 25 was needed only to initialize the widget.



3. Display and filter records

Replace `app.py` with:

```
import pandas as pd
import streamlit as st

st.title("A small movie collection")
movies = [
    {"title": "Arrival", "year": 2016, "rating": 7.9},
    {"title": "Spirited Away", "year": 2001, "rating": 8.6},
    {"title": "The Iron Giant", "year": 1999, "rating": 8.1},
]

minimum = st.slider("Minimum rating", 0.0, 10.0, 7.0, 0.1)
selected = [movie for movie in movies if movie["rating"] >= minimum]

st.metric("Matching movies", len(selected))
st.dataframe(pd.DataFrame(selected), hide_index=True)
```

The list comprehension performs the query. The table only displays its result. Changing the interface library would not change the meaning of that query.

Try it: Add a title search with `st.text_input()` and combine it with the rating filter. Make the search case-insensitive.

The same pattern with a DataFrame as the value

A control can return an entire table rather than one number. Use `st.data_editor()` when the user should edit a table and your Python code should receive the edited data. When given a pandas DataFrame, it returns a pandas DataFrame.

Replace `app.py` with:

```
import pandas as pd
import streamlit as st

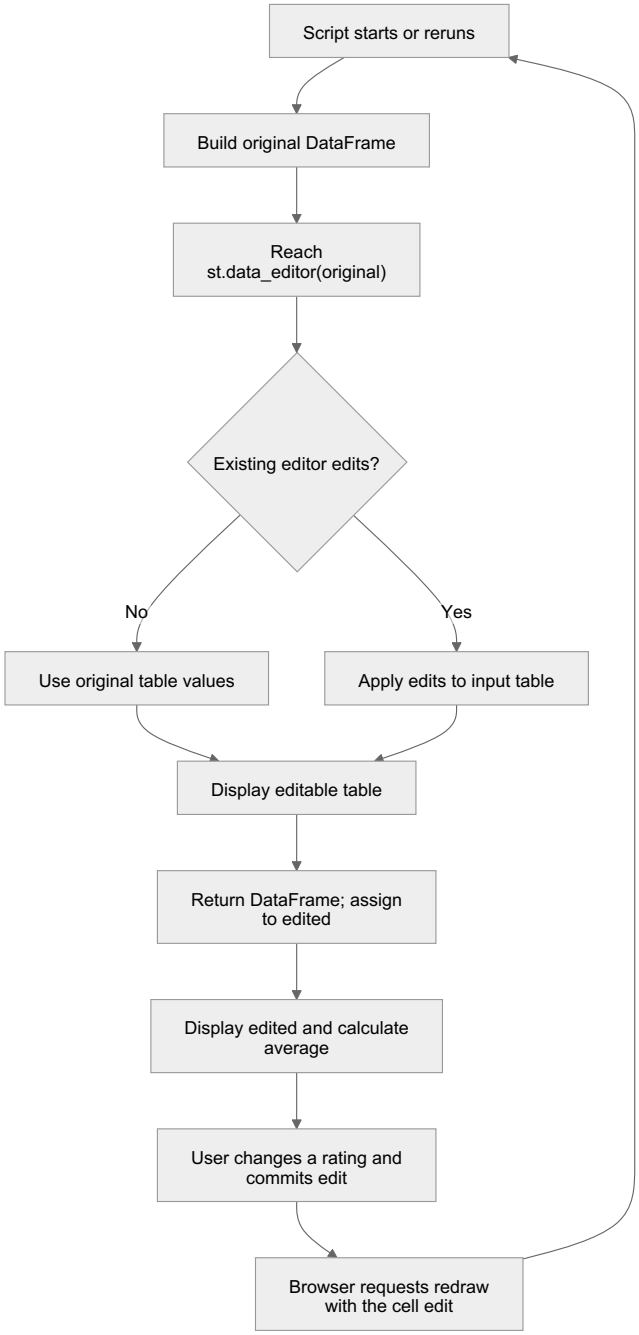
st.title("Edit a movie collection")

original = pd.DataFrame([
    {"title": "Arrival", "rating": 7.9},
    {"title": "Spirited Away", "rating": 8.6},
    {"title": "The Iron Giant", "rating": 8.1},
])

edited = st.data_editor(
    original,
    key="movie_editor",
    hide_index=True,
    disabled=["title"],
    column_config={
        "rating": st.column_config.NumberColumn(
            "Rating", min_value=0.0, max_value=10.0, step=0.1
        )
    },
)

st.write("The DataFrame returned by the editor:")
st.dataframe(edited, hide_index=True)
st.write("Average rating:", round(edited["rating"].mean(), 2))
```

Initially, `edited` contains the original values. Change Arrival's rating from 7.9 to 9.0 and commit the cell edit by pressing Enter. The script reruns, the editor displays the changed rating, and the call returns a DataFrame containing 9.0. The table and average below it use that returned DataFrame.



Notice that `original` is rebuilt on each run with the same starting data. Streamlit retains the editor's changes and uses them to produce the returned table. The assignment to `edited` runs again each time; the local variable itself is not surviving between runs.

The returned table does not automatically modify `original` or write a file. Saving is a separate application operation, which we will add in the CRUD example.

The two table functions have different purposes:

Call	Purpose	Return value used here
`st.dataframe(data)`	Display a table	Not an edited DataFrame
`st.data_editor(data)`	Display an editable table	Edited data; a DataFrame when the input is a DataFrame

Try changing two ratings in succession. Observe that both edits appear in the returned table and contribute to the average.

Editor documentation: https://docs.streamlit.io/develop/api-reference/data/st.data_editor

4. Preserve values between reruns

Replace `app.py` with:

```
import streamlit as st

st.title("A counter")
if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Add one"):
    st.session_state.count += 1

st.write("Count:", st.session_state.count)
```

`st.session_state` holds values across reruns for a browser session. A new session gets its own state. It is not permanent storage: restarting the server or losing the session can discard it.

5. Reduce unnecessary work

Cache a computed result

Replace `app.py` with:

```
import time
import streamlit as st

@st.cache_data
def slow_total(limit):
    time.sleep(2)  # Simulate expensive work.
    return sum(range(limit + 1))

st.title("Caching a calculation")
limit = st.slider("Upper limit", 1, 100, 10)
st.write("Total:", slow_total(limit))
st.checkbox("An unrelated control")
```

The first call for a particular input pauses. Repeating that call can reuse the result. Change the checkbox: the script reruns, but the expensive calculation need not run again. Change the slider to a new input and observe the delay.

Use `st.cache_data` for reusable computed data, such as parsed files and query results. Its returned values are copies, so changing a returned list does not update the cached value. Use `st.cache_resource` for shared resources such as database connections; shared mutable resources require care with concurrent access. We will use session state for our editable collection.

Caching is not persistence. A cached result can disappear and be recomputed. A saved file should remain after the application stops.

Cache a partial DataFrame loaded from JSON

Suppose we want only movies whose titles contain a search phrase and whose ratings meet a minimum. The cached function will read the local JSON file, select that subset, and return a DataFrame containing only the matches.

First complete the download and preparation instructions in Section 6. They create `data/movies.json` with the normalized fields `id`, `title`, `genre`, and `rating`. Then return here and replace `app.py` with:

```
from pathlib import Path

import pandas as pd
import streamlit as st

DATA_PATH = Path(__file__).resolve().parent / "data" / "movies.json"


@st.cache_data(max_entries=1)
def get_movie_subset(filename, title_contains, minimum_rating):
    print("Reading JSON and selecting movies:", title_contains, minimum_rating)
    movies = pd.read_json(filename)
    title_matches = movies["title"].str.contains(
        title_contains, case=False, regex=False, na=False
    )
    rating_matches = movies["rating"].ge(minimum_rating)
    return movies.loc[title_matches & rating_matches].copy()


st.title("Cache a selected set of movies")

with st.form("search"):
    title_contains = st.text_input("Title contains", value="star")
    minimum_rating = st.slider("Minimum rating", 0.0, 10.0, 7.0, 0.1)
    st.form_submit_button("Search")

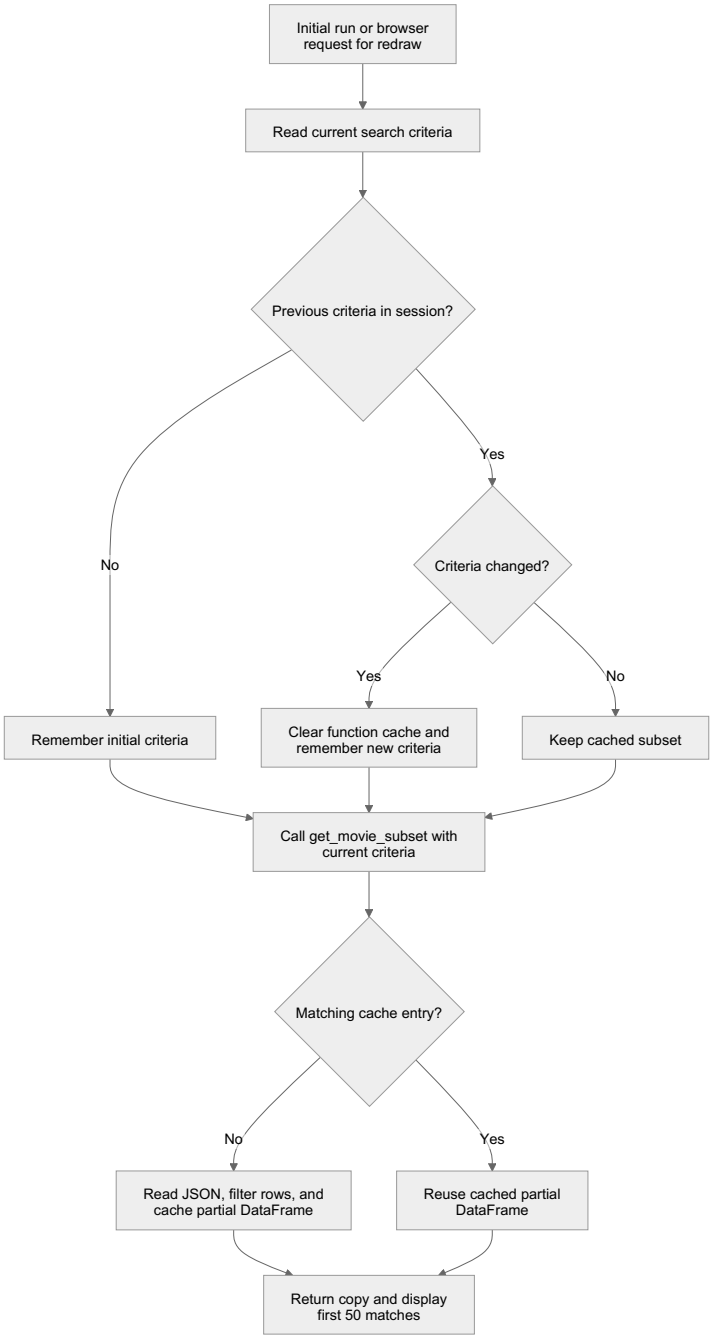
# Treat leading/trailing whitespace and letter case as insignificant.
criteria = (title_contains.strip().casefold(), minimum_rating)

if "previous_movie_criteria" not in st.session_state:
    st.session_state.previous_movie_criteria = criteria
elif criteria != st.session_state.previous_movie_criteria:
    get_movie_subset.clear()
    st.session_state.previous_movie_criteria = criteria
    st.info("Search criteria changed: cleared the cached subset.")

try:
    matching = get_movie_subset(str(DATA_PATH), *criteria)
except (OSError, ValueError, KeyError) as error:
    st.error(f"Cannot load the movie subset: {error}")
    st.info("Complete the dataset preparation in Section 6 first.")
    st.stop()

st.write(f"{len(matching):,} matching movies")
st.dataframe(matching.head(50), hide_index=True)
st.checkbox("An unrelated control")
```

On the initial run, the defaults request titles containing “star” with a rating of at least 7.0. Missing ratings do not satisfy the minimum. The browser holds subsequent changes to these two controls until you press Search. It then sends both values together in one request for a redraw.


Initial run or browser request for redraw
Read current search criteria
Previous criteria in session?
No
Yes
Criteria changed?
Yes
No
Remember initial criteria
Clear function cache and remember new criteria
Keep cached subset
Call get_movie_subset with current criteria
Matching cache entry?
No
Yes
Read JSON, filter rows, and cache partial DataFrame
Reuse cached partial DataFrame
Return copy and display first 50 matches

Follow one search through the application

Event	Criteria	Cache action	JSON read?
Initial run with an empty cache	Title contains “star”; rating at least 7.0	Build and cache that subset	Yes
Click the unrelated checkbox	Same criteria	Reuse that subset	No
Change text to “moon,” then press Search	Title contains “moon”; rating at least 7.0	Explicitly clear cache; build new subset	Yes
Change minimum to 8.0, then press Search	Title contains “moon”; rating at least 8.0	Explicitly clear cache; build new subset	Yes
Press Search with unchanged criteria	Same criteria	Reuse that subset	No

The table assumes a single active session, no source-code changes, and no other cache clearing.

The function returns the **whole matching subset**, although the interface displays only its first 50 rows. On a cache miss, `pd.read_json()` still reads the entire JSON array into memory before filtering. Caching the subset avoids repeating that work for the same search; it does not give a JSON file indexed retrieval.

The comparison with `previous_movie_criteria` decides whether to discard the previous result. The call to `get_movie_subset.clear()` clears this function’s cached entries. It does not clear widget state, other functions’ caches, or the JSON file. The next function call then reads the file and computes the new subset.

Explicit clearing versus an automatic cache miss

The criteria are also function arguments, so Streamlit already distinguishes searches by their argument values. **Changing the arguments is sufficient to cause a cache miss for a subset that is not cached.** Explicit clearing is included here to demonstrate deliberately discarding the old subset when the criteria change; it is not required for correct filtering. `max_entries=1` also limits this function to one cached result.

Without the explicit clearing and with room for several cache entries, changing from “star” to “moon” and back to “star” could reuse the earlier “star” result. This example instead discards the old result on each criteria change.

The cache does not automatically detect changes to the JSON file’s contents from the filename argument. If the file changes while the criteria stay the same, clear the function’s cache after saving, pass a file-version value as another argument, or use a suitable TTL. The later CRUD example uses session state and explicit file writes rather than this read-only subset cache.

By default, data caches can be shared across sessions, whereas `previous_movie_criteria` is per session. This demonstration assumes one active session; another session clearing this function’s cache could cause an additional read.

Watch the terminal while performing the events in the table. The print message appears only when the function body runs, making cache hits and misses observable.

Caching documentation: https://docs.streamlit.io/develop/concepts/architecture/caching

Batch input with a form

Replace `app.py` with:

```
import streamlit as st

st.title("Submit several values together")
with st.form("movie_form"):
    title = st.text_input("Movie title")
    year = st.number_input("Release year", 1888, 2100, 2020)
    submitted = st.form_submit_button("Submit")

if submitted:
    st.write({"title": title, "year": year})
```

A form collects input changes and submits them together. This avoids rerunning the application for each individual change inside the form.

Other useful optimizations

- Filter before displaying; avoid sending the entire collection to the browser.
- Display a bounded page of results.
- Load files once per session when maintaining an editable in-memory collection.
- Save only after a successful operation actually changes data.
- Measure before optimizing. Caching a trivial expression may cost more than computing it.

6. Get the dataset

We will use the Vega movie dataset, which contains thousands of movie records. Download it once; the application will work with a local copy. The original has additional fields, missing values, and some titles represented as numbers. We will normalize only the fields we need.

Dataset information: https://vega.github.io/vega-datasets/datasets/movies/

Download: https://vega.github.io/vega-datasets/data/movies.json

Create `download_data.py`:

```
from pathlib import Path
from urllib.request import urlretrieve

folder = Path(__file__).resolve().parent / "data"
folder.mkdir(exist_ok=True)
destination = folder / "movies-source.json"

if destination.exists():
    print("Source file already exists:", destination)
else:
    urlretrieve(
        "https://vega.github.io/vega-datasets/data/movies.json",
        destination,
    )
    print("Downloaded:", destination)
```

Run it:

```
python download_data.py
```

If your environment blocks the download, open the download link in a browser and save the file as `data/movies-source.json`.

Make a large working collection

The original dataset is useful, but modest in size. For this exercise we will produce 100,000 records by repeating and normalizing the source records. These are deliberately duplicated teaching records, not 100,000 distinct real movies. Each copy has its own stable identifier.

Create `prepare_data.py`:

```
import json
from pathlib import Path

folder = Path(__file__).resolve().parent / "data"
source = folder / "movies-source.json"
target = folder / "movies.json"

if target.exists():
    raise SystemExit("movies.json already exists; preparation will not overwrite it.")

with source.open(encoding="utf-8") as file:
    original = json.load(file)

if not isinstance(original, list) or not original:
    raise SystemExit("Expected a nonempty JSON array.")

movies = []
for index in range(100_000):
    row = original[index % len(original)]
    raw_rating = row.get("IMDB_Rating")
    rating = None if raw_rating is None else float(raw_rating)
    movies.append({
        "id": f"movie-{index + 1:06d}",
        "title": str(row.get("Title") or "Untitled"),
        "genre": str(row.get("Major_Genre") or "Unknown"),
        "rating": rating,
    })

with target.open("w", encoding="utf-8") as file:
    json.dump(movies, file, ensure_ascii=False, indent=2)

print(f"Created {len(movies):,} records in {target}")
print(f"File size: {target.stat().st_size / 1_000_000:.1f} MB")
```

Run it:

```
python prepare_data.py
```

Our “giant JSON file” is one JSON array. Python loads the entire array into a list of dictionaries. It is not JSON Lines, and we will not read one record at a time.

A record looks like:

```
{
  "id": "movie-000001",
  "title": "Example Movie",
  "genre": "Drama",
  "rating": 7.5
}
```

A JSON `null` rating becomes Python `None`. A missing rating is different from a rating of zero.

7. Separate storage operations from the interface

Create `storage.py`:

```
import json
import os
import tempfile
from pathlib import Path

DATA_PATH = Path(__file__).resolve().parent / "data" / "movies.json"


def load_movies(path=DATA_PATH):
    with path.open(encoding="utf-8") as file:
        movies = json.load(file)

    if not isinstance(movies, list):
        raise ValueError("The data file must contain a JSON array.")

    seen = set()
    for movie in movies:
        if not isinstance(movie, dict):
            raise ValueError("Every record must be an object.")
        identifier = movie.get("id")
        if not isinstance(identifier, str) or not identifier or identifier in seen:
            raise ValueError("Record IDs must be nonempty, unique strings.")
        seen.add(identifier)
        if not isinstance(movie.get("title"), str):
            raise ValueError("Each movie needs a string title.")
        if not isinstance(movie.get("genre"), str):
            raise ValueError("Each movie needs a string genre.")
        rating = movie.get("rating")
        if rating is not None and (
            isinstance(rating, bool)
            or not isinstance(rating, (int, float))
            or not 0 <= rating <= 10
        ):
            raise ValueError("Ratings must be null or numbers from 0 to 10.")
    return movies


def save_movies(movies, path=DATA_PATH):
    # Write beside the destination so replacement stays on one filesystem.
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = None
    try:
        with tempfile.NamedTemporaryFile(
            mode="w", encoding="utf-8", dir=path.parent,
            prefix="movies-", suffix=".tmp", delete=False,
        ) as file:
            temporary = Path(file.name)
            json.dump(movies, file, ensure_ascii=False, indent=2, allow_nan=False)
            file.flush()
            os.fsync(file.fileno())
        os.replace(temporary, path)
    finally:
        if temporary is not None:
            temporary.unlink(missing_ok=True)
```

A temporary file followed by replacement avoids exposing a half-written JSON document during ordinary operation. It does not provide database transactions, conflict detection, or protection against every storage failure.

We keep file I/O outside the UI so it can be tested and later replaced by database operations.

8. Build the complete CRUD application

CRUD means **Create, Read, Update, Delete**. Replace all of `app.py` with the following complete program. Keep `storage.py` in the same directory.

```python
from uuid import uuid4

import pandas as pd
import streamlit as st

from storage import load_movies, save_movies

st.set_page_config(page_title="Movie collection", layout="wide")
st.title("Movie collection")
st.caption("One local JSON file, loaded into memory for this session.")

if "movies" not in st.session_state:
    try:
        st.session_state.movies = load_movies()
    except (OSError, ValueError) as error:
        st.error(f"Cannot load movies: {error}")
        st.info("Run download_data.py and prepare_data.py first.")
        st.stop()

if "notice" in st.session_state:
    st.success(st.session_state.pop("notice"))


def commit(candidate, message):
    # Save first. Publish the new in-memory collection only after success.
    try:
        save_movies(candidate)
    except (OSError, ValueError, TypeError) as error:
        st.error(f"Save failed; the in-memory collection was not changed: {error}")
        return
    st.session_state.movies = candidate
    st.session_state.notice = message
    st.rerun()


movies = st.session_state.movies
st.metric("Total records", f"{len(movies):,}")

# READ: search in memory and send only a page to the browser.
st.subheader("Read: find movies")
query = st.text_input("Title contains").strip().casefold()
minimum = st.slider("Minimum rating", 0.0, 10.0, 0.0, 0.1)
include_unknown = st.checkbox("Include movies without ratings", value=True)

matches = [
    movie for movie in movies
    if query in movie["title"].casefold()
    and (
        (movie["rating"] is None and include_unknown)
        or (movie["rating"] is not None and movie["rating"] >= minimum)
    )
]
st.write(f"{len(matches):,} matching records")

page_size = 50
page_count = max(1, (len(matches) + page_size - 1) // page_size)
# This input has no dynamic maximum; clamp the requested page below.
requested_page = int(st.number_input("Page", min_value=1, value=1, step=1))
page = min(requested_page, page_count)
if requested_page != page:
    st.caption(f"Showing the last available page: {page}.")
start = (page - 1) * page_size
```

```
visible = matches[start:start + page_size]
st.dataframe(
    pd.DataFrame(visible, columns=["id", "title", "genre", "rating"]),
    hide_index=True,
)
st.caption(f"Page {page} of {page_count}. Copy a record ID to edit or delete it.")

# CREATE: a form batches inputs, and blank titles are rejected.
st.subheader("Create: add a movie")
with st.form("create_movie", clear_on_submit=True):
    new_title = st.text_input("Title", key="new_title")
    new_genre = st.text_input("Genre", value="Unknown", key="new_genre")
    new_rating = st.number_input(
        "Rating", min_value=0.0, max_value=10.0, value=5.0, step=0.1,
        key="new_rating",
    )
    new_unknown = st.checkbox("Rating is unknown", key="new_unknown")
    create = st.form_submit_button("Create movie")

if create:
    if not new_title.strip():
        st.error("A title is required.")
    else:
        record = {
            "id": str(uuid4()),
            "title": new_title.strip(),
            "genre": new_genre.strip() or "Unknown",
            "rating": None if new_unknown else float(new_rating),
        }
        commit(movies + [record], "Movie created and saved.")

# UPDATE and DELETE use stable IDs, not positions or possibly repeated titles.
st.subheader("Update or delete a movie")
identifier = st.text_input("Record ID").strip()
selected = next((movie for movie in movies if movie["id"] == identifier), None)

if identifier and selected is None:
    st.warning("No movie has that ID.")

if selected is not None:
    st.json(selected)
    # Including the ID in the form and widget keys resets inputs when
    # the user selects a different record.
    with st.form(f"edit_{identifier}"):
        edited_title = st.text_input(
            "Title", value=selected["title"], key=f"title_{identifier}"
        )
        edited_genre = st.text_input(
            "Genre", value=selected["genre"], key=f"genre_{identifier}"
        )
        edited_rating = st.number_input(
            "Rating", min_value=0.0, max_value=10.0,
            value=float(selected["rating"] or 0.0), step=0.1,
            key=f"rating_{identifier}",
        )
        edited_unknown = st.checkbox(
            "Rating is unknown", value=selected["rating"] is None,
            key=f"unknown_{identifier}",
        )
        update = st.form_submit_button("Save changes")

    if update:
        if not edited_title.strip():
```

```
            st.error("A title is required.")
        else:
            replacement = {
                **selected,
                "title": edited_title.strip(),
                "genre": edited_genre.strip() or "Unknown",
                "rating": None if edited_unknown else float(edited_rating),
            }
            if replacement == selected:
                st.info("Nothing changed; no file was written.")
            else:
                candidate = [
                    replacement if movie["id"] == identifier else movie
                    for movie in movies
                ]
                commit(candidate, "Movie updated and saved.")

    with st.form(f"delete_{identifier}"):
        confirmed = st.checkbox("I confirm deletion of this movie")
        delete = st.form_submit_button("Delete movie")
    if delete:
        if not confirmed:
            st.error("Confirm deletion first.")
        else:
            candidate = [movie for movie in movies if movie["id"] != identifier]
            commit(candidate, "Movie deleted and saved.")
```

Run it:

```
python -m streamlit run app.py
```

The app loads the JSON collection once per session. Reading, searching, changing a page, or submitting an unchanged edit does not write the file. Successful creates, changed updates, and confirmed deletions write the complete collection. The next rerun uses the updated in-memory list.

`commit()` builds on an important ordering rule: prepare a candidate collection, save it, then replace the session's collection. We avoid mutating existing dictionaries before saving. A failed save therefore leaves the session's current collection intact.

`st.rerun()` stops the current run and starts another. That refreshes the count and table, which appear earlier in the script. The success message survives that rerun through session state.

Why not cache the editable collection?

A zero-argument cached file loader can return stale data after the file changes: the filename alone does not describe its contents. `st.cache_data` also returns copies, so mutating a returned value does not persist an edit. Our CRUD app instead owns a session-local collection and explicitly writes changes.

For a separate **read-only** application, you could cache parsing and include the file's modification information as cache inputs:

```
import json
from pathlib import Path
import streamlit as st

@st.cache_data(max_entries=2)
def read_snapshot(filename, modified_ns, size):
    # The metadata arguments participate in the cache key.
    with open(filename, encoding="utf-8") as file:
        return json.load(file)

path = Path(__file__).resolve().parent / "data" / "movies.json"
metadata = path.stat()
movies = read_snapshot(str(path), metadata.st_mtime_ns, metadata.st_size)
```

This is a practical cache invalidation technique, not a concurrency protocol or a guaranteed content fingerprint. Do not insert it into the CRUD app as a second competing owner of editable data.

9. Verify the behavior

Perform these checks with only one app session open:

1. Search for a title and confirm the count and table change.
2. Create a uniquely named movie. Find it and copy its ID.
3. Change its rating and submit. Confirm the new value appears.
4. Submit the same values again. Confirm the app reports no change.
5. Stop and restart the server, then open a fresh browser session. Find the movie again. The saved edit should remain.
6. Try deleting without confirmation. The movie should remain.
7. Confirm and delete it. Restart in a fresh session and confirm it is absent.
8. Try creating a blank title. The collection should not change.
9. Filter until there are no matches. The empty result should display normally.

To inspect the saved collection independently, enter this as `inspect_data.py`:

```
from storage import load_movies

movies = load_movies()
print("Records:", len(movies))
print("Unique IDs:", len({movie["id"] for movie in movies}))
print("First record:", movies[0] if movies else "No records")
```

```
python inspect_data.py
```

To check that reads do not write, compare the modification timestamp printed by this command before and after searching or changing pages:

```
python -c "from storage import DATA_PATH; print(DATA_PATH.stat().st_mtime_ns)"
```

10. What this design teaches - and where it stops

This is a **single-user, local teaching application**. Use one active session and one server process. Two sessions can load different snapshots; a later save can overwrite another session's changes. Atomic replacement prevents partial replacement, but does not prevent lost updates.

The whole dataset must fit in memory. Loading JSON, scanning for matches, copying lists, and rewriting the file all become more expensive as it grows. Pagination reduces browser output; it does not eliminate the in-memory scan. The prepared data contains synthetic duplicates specifically so these costs are visible.

A local file must live on writable, persistent storage if edits are to survive deployment restarts. Some hosting environments discard local files. The application also has no authentication or authorization.

These limitations lead naturally to database topics: keys, indexes, constraints, transactions, isolation, and durable storage. A database can retrieve a page without loading every record, update one row without rewriting every record, and coordinate competing writers.

Exercises

1. Add a genre filter and a sort order. Explain which operations still scan all records.
2. Add a reset-page button when search conditions change.
3. Measure load, search, and save times using `time.perf_counter()`. Compare 10,000 and 100,000 records.
4. Add a rule that prevents duplicate title-and-genre pairs. Decide how the deliberately duplicated starting data affects that rule.
5. Add an ID-to-record dictionary for faster individual lookups. Keep it consistent after each successful change.
6. Replace JSON storage with SQLite. Preserve the CRUD interface while moving filtering and pagination into SQL.

References

- Streamlit installation and first steps: https://docs.streamlit.io/get-started
- Execution and reruns: https://docs.streamlit.io/develop/concepts/architecture
- Session state: https://docs.streamlit.io/develop/concepts/architecture/session-state
- Forms: https://docs.streamlit.io/develop/concepts/architecture/forms
- Caching: https://docs.streamlit.io/develop/concepts/architecture/caching
- Vega datasets and source attribution: https://github.com/vega/vega-datasets

The movie data is third-party sample data distributed by Vega; consult its dataset metadata and original source terms before redistributing it beyond the exercise.